Testing code

Paco

2025-11-18

Table of contents

1 Bagging

1.1 Bagging: main idea

- **Bootstrap aggregation**, or **bagging**, is a general-purpose procedure for reducing the variance of a statistical learning method. It reduces variability by training multiple models on different bootstrap samples and averaging their predictions. Since each model sees a slightly different subset of the data, their individual errors tend to cancel out when aggregated, leading to a smoother and more stable estimator.
- Given a set of B independent observations $z_1, \dots, z_B$ each with variance σ^2, the variance of the mean

$$\bar{z} = \frac{1}{B}\sum_{i=1}^{B} z_i$$

is σ^2/B.

- We can therefore reduce the variance by averaging a set of predictions. However, this is not practical because we usually do not have access to multiple training sets.
- Instead, we use the bootstrap by taking repeated samples from the training data

$$Z = \{(x_i, y_i),\ i = 1, \dots, n\}.$$

Bagging works by creating multiple bootstrap samples from the original dataset, training a separate model on each, and averaging their predictions. This aggregation stabilises the model by lowering variance without significantly increasing bias.

1.2 The bootstrap

- Let B be the number of desired bootstrap datasets $Z^{*1}, \dots, Z^{*B}$.
- For $b = 1, \dots, B$:
  - Sample n observations from Z **with replacement**.
  - This is the bth bootstrap dataset, denoted Z^{*b}.
- Each bootstrap dataset contains on average 63.2% of the original samples.

The bootstrap procedure generates multiple resampled datasets by drawing with replacement from the original data. Each sample replicates the variability inherent in the population, allowing us to estimate model variance or prediction stability without requiring new data. This approach underlies bagging and several ensemble methods.

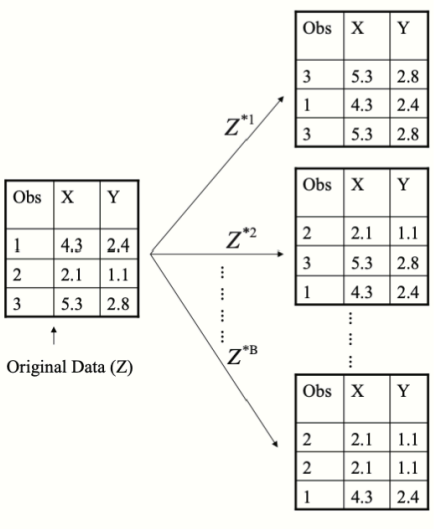

Figure 1: Bootstrap sampling

1.3 Bagging

- Let $Z^{*1}, \dots, Z^{*B}$ be B bootstrap datasets from the training data. For all $b = 1, \dots, B$, we can fit our predictive regression model $\hat{f}^{*b}$ on the bth bootstrap dataset Z^{*b}.

- We then average all the predictions to obtain a single bagged model

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B}\sum_{b=1}^{B} \hat{f}^{*b}(x)$$

  which is called **bagging**. In other words, we take all the B predictions made by the B models trained on different bootstrap samples. Then average those predictions to get a final one ($\hat{f}_{\text{bag}}(x)$).
- If the individual models $\hat{f}^{*b}$ have low bias but high variance, the combined estimator $\hat{f}_{\text{bag}}$ retains a similar bias but exhibits lower variance.
- In practice, we use a value of B sufficiently large for the error to stabilise. Typically, a value between 100 and 1000 is sufficient.

Bagging therefore reduces variance through model aggregation while preserving the bias of the underlying learners, resulting in more stable and robust predictions.

1.4 Out-of-bag error estimate

- There is a straightforward way to estimate the test error of a bagged model, called the **out-of-bag (OOB) error estimate**.
- The key idea of bagging is that the model is repeatedly fitted to bootstrap subsets of the data, which contain on average about two-thirds of the observations (≈ 63%).
- The remaining one-third (≈ 37%) of the observations not used to fit a given bagged model are referred to as the **out-of-bag observations**.
- We can predict the response for the ith observation using all models $\hat{f}^{*b}$ with $b \in I_{x_i} = \{b = 1, \dots, B : (x_i, y_i) \notin Z^{*b}\}$, i.e. the models for which the observation was OOB. This yields around $B/3$ predictions for the ith observation, which we average:

$$\text{Err}_{\hat{f}_{\text{bag}}}(x_i) = \frac{1}{|I_{x_i}|}\sum_{b \in I_{x_i}} (y_i - \hat{f}^{*b}(x_i))^2.$$

  In other words, this is *the mean of the squared prediction errors made by all the models* b in I_{x_i} (the models for which x_i was OOB).
- The average over all n observations gives the overall OOB error estimate:

$$\text{Err}_{\hat{f}_{\text{bag}}} = \frac{1}{n}\sum_{i=1}^{n} \text{Err}_{\hat{f}_{\text{bag}}}(x_i).$$

  On average, how well does the bagged model generalise to unseen data?
- For large B, this estimator can be shown to be equivalent to the Leave-One-Out Cross-Validation (LOOCV) estimator, but in bagging, it comes for free.

The OOB error provides an internal validation mechanism without requiring an explicit test set, making it especially efficient for ensemble methods such as Random Forests.

1.5 Bagging for trees

- Classification and regression trees are perfectly suited for bagging, since their poor predictive accuracy comes from their high variance.
- For a regression problem, [1] for all B bootstrap training sets we grow a large tree **without pruning** to obtain a high-variance predictive model. We then average all these (possibly overfitted) trees to obtain a bagged model with reduced variance.
- This has shown to give impressive improvements in prediction accuracy, though at the cost of interpretability.
- Importantly, B is not a classical tuning parameter, since a large B will not overfit the data; the error will simply stabilise.
- For classification, we fit B classification trees $\hat{G}^{*b}$ and then choose the predicted class by majority vote, that is

$$\hat{G}_{\text{bag}}(x) = \arg\max_{g=1,\ldots,q} \frac{1}{B} \sum_{b=1}^{B} \mathbf{1}\{g = \hat{G}^{*b}(x)\}$$

  - $\hat{G}_{\text{bag}}(x)$: the **final bagged classifier**, i.e. the class label the **ensemble of trees** assigns to x after combining all their votes.
  - $\arg\max_{g=1,\ldots,q}$: we look across all possible classes and choose the one with the **largest average vote** from the trees.
  - $\frac{1}{B}$: we compute the **fraction of trees** that predict each class g.
  - $\hat{G}^{*b}(x)$: the **prediction of the** bth tree for observation x.
  - $\mathbf{1}\{g = \hat{G}^{*b}(x)\}$: for a given class g, this equals 1 if the bth tree predicts class g, and 0 otherwise. The sum $\sum_{b=1}^{B} \mathbf{1}\{g = \hat{G}^{*b}(x)\}$ therefore counts how many trees voted for g.
- Bagging is therefore especially effective for high-variance, low-bias models such as decision trees, as averaging across bootstrap replicates substantially reduces their prediction variance.

1.6 Example: bagging and variance reduction

To illustrate the effect of bagging, we return to the **heart disease** dataset, where decision trees are used to predict whether a patient has heart disease. As seen earlier, a single unpruned tree has high variance: small changes in the data can lead to very different tree structures.

Bagging mitigates this instability by repeatedly resampling the training data and averaging the resulting models.

In Figure 2, three trees are trained on different bootstrap samples drawn **with replacement** from the same dataset. Each tree follows a different structure and makes slightly different predictions, showing the variability of high-variance learners such as decision trees.

[1] In this context, a regression problem refers to a supervised learning task where the target variable y is continuous. In contrast, a classification problem deals with categorical outputs.

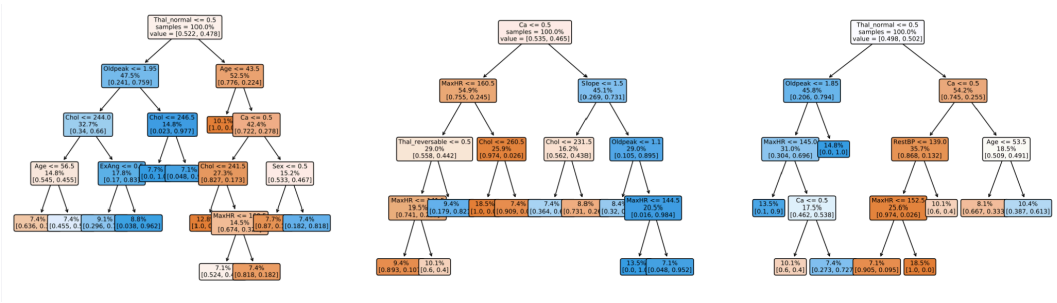

Figure 2: Bootstrapped trees

By averaging the predictions from all these trees, bagging produces a single, more stable model with reduced variance. The gain from aggregation is evident in the out-of-bag (OOB) error plot below.

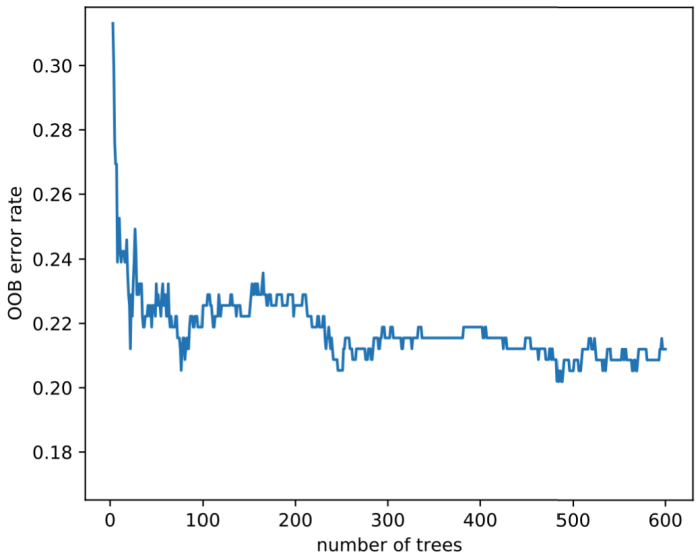

Figure 3: OOB error rate for bagged trees

In Figure 3, the **OOB error rate** decreases rapidly as the number of trees increases, then stabilises once variance reduction saturates. This shows how bagging improves generalisation performance by averaging across many slightly different models rather than relying on a single, unstable one.

2 Random forest

2.1 Variance reduction and correlation

- Recall that given a set of B **independent** observations $z_1, \ldots, z_B$, each with variance σ^2, the variance of their mean is

$$\bar{z} = \frac{1}{B} \sum_{i=1}^{B} z_i$$

  and the variance of $\bar{z}$ is σ^2/B.
- This result holds only if the z_i are independent. If they are **correlated**, the variance reduction achieved by averaging becomes much smaller.
- In bagging, each model is trained on a bootstrap sample of the data. These samples overlap substantially, so the fitted trees are often **correlated**. As a result, the averaging in bagging does not reduce variance as effectively as it would if the models were completely independent.
- To understand this, consider $z_i \sim \mathcal{N}(0, 1)$ and three levels of correlation between them:

  1. **Independence:** $\text{cor}(z_i, z_j) = 0$ for all i, j.
  2. **Weak dependence:** $\text{cor}(z_i, z_{i+1}) = 0.3$.
  3. **Strong dependence:** $\text{cor}(z_i, z_{i+1}) = 0.8$.

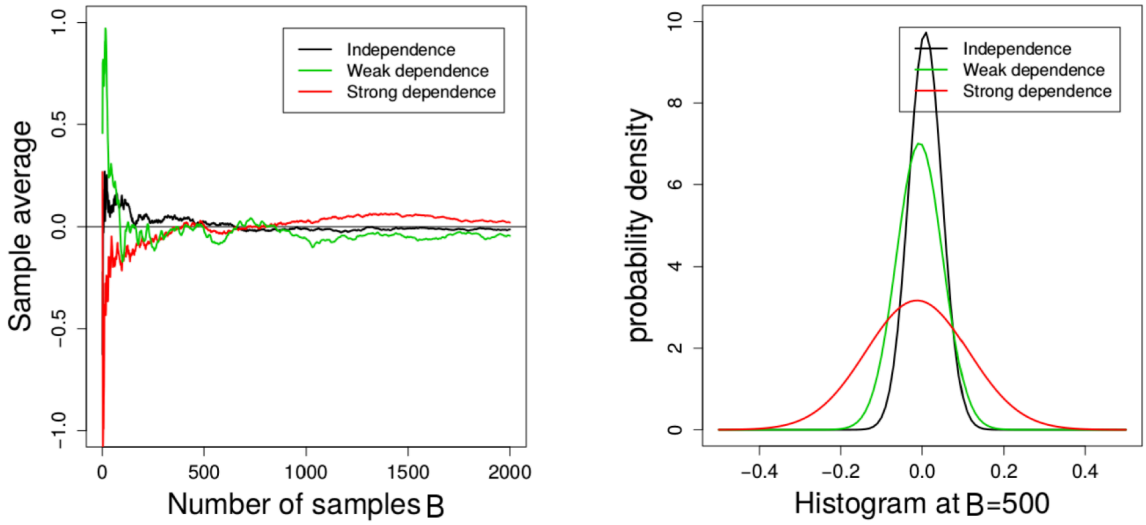

Figure 4: Effect of Correlation on Variance Reduction

As seen in Figure 4, correlation among observations (or models) greatly affects how quickly variance decreases when averaging.

The **left plot** shows the sample average as the number of samples B increases. When the observations are independent (black line), the average quickly stabilises near zero. With weak (green) or strong (red) dependence, convergence is much slower and the variability remains higher.

The **right plot** shows the distribution of the sample mean when $B = 500$. Independence (black) yields a narrow distribution with small variance; as correlation increases, the distribution widens, meaning the average fluctuates more.

This demonstrates that averaging only reduces variance effectively when the individual models or samples are **uncorrelated**. If the models are correlated (because they rely on similar features or data points) the benefit of averaging diminishes.

In the next section, we see how random forests address this limitation by introducing randomness in feature selection (by randomly choosing subsets of predictors at each split) which reduces correlation between trees and improves variance reduction beyond bagging.

```
#TODO add this foot ntoe_ the specific mechanism in random forests where, at each split of a
```

2.2 Bagging: correlated trees

- In bagging, each tree is trained on a bootstrap dataset that is drawn **with replacement** from the original training data. Because these bootstrap samples overlap substantially, the trees share **many common observations**.
- The decision tree algorithm is **greedy**: it chooses at each split the variable that gives the largest reduction in impurity. Since the datasets are very similar, the trees tend to select the **same predictors for their early splits**.

```
#TODO include this as a footnote: Impurity quantifies how mixed the node is. The decision tre
# In classification, impurity is measured using the Gini index or entropy, while in regressio
```

- As a result, the trees end up looking very similar and their predictions $\hat{f}^{*b}(x_0)$ for a new input x_0 are **highly correlated** for all $b = 1, \dots, B$. In other words, for all $b = 1, \dots, B$" means that this correlation property applies across all the trees in the bagged ensemble, each $\hat{f}^{*b}(x_0)$ tends to give a similar prediction for the same input x_0.

As seen in Figure 2, the three trees trained on different bootstrap samples from the same dataset share much of their structure.
They all split early on similar variables (Thal, Ca, Oldpeak), showing that even after resampling, the bagged trees remain strongly correlated.This shows how, despite resampling, the bagged trees remain **strongly correlated** because they repeatedly rely on the same dominant predictors for their early decisions.

Such correlation limits the benefit of averaging in bagging: even though we combine many trees, if they make **similar errors**, the ensemble's variance does not decrease as much as expected.

This motivates the next step, i.e., **random forests**, which introduce randomness in feature selection at each split to decorrelate the trees and achieve greater variance reduction.

```
#TODO but if we add randomness to then reduce the variance, isnt it a bit working twice?
```

2.3 Random forest: main idea

- **Random forests** extend the concept of bagging. The key idea is to **decorrelate** the trees by introducing randomness during the growing process.
- In bagging, trees are highly correlated because they often use the same strong predictors for their first splits. Random forests reduce this correlation by **randomly selecting a subset of predictors** at each split.
- This randomisation prevents all trees from relying on the same dominant variables, leading to a more diverse ensemble and stronger variance reduction.

2.3.1 Random forest algorithm

1. **Bootstrap the data** B times, obtaining B bootstrap samples $Z^{*1}, \dots, Z^{*B}$.
2. **Grow a tree** on each bootstrap sample. At each node, randomly select m predictors out of p, and choose the best split among those m.
3. **Grow the trees fully**, without pruning (as in bagging).
4. **Aggregate predictions**:
    - For **regression**, average the predictions from the B trees.
    - For **classification**, use the **majority vote** among the B trees.

2.3.2 Intuition and choice of m

As seen in Figure 5, selecting only a random subset of variables at each split prevents trees from becoming too similar.
Even though all trees are trained on overlapping bootstrap samples, their different variable choices make them less correlated, which in turn improves variance reduction and generalisation performance.

Typical values for m:

- For **classification**, $m \approx \sqrt{p}$.
- For **regression**, $m \approx p/3$.

This rule of thumb works well in practice because it balances diversity and predictive strength: small m increases randomness (reducing correlation), while large m allows trees to use stronger predictors (reducing bias).

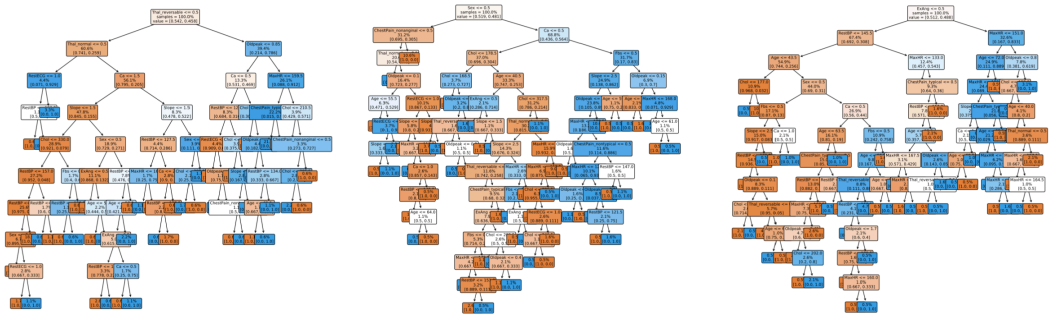

Figure 5: Random forest trees

2.4 Heart data: bagging and random forest in Python

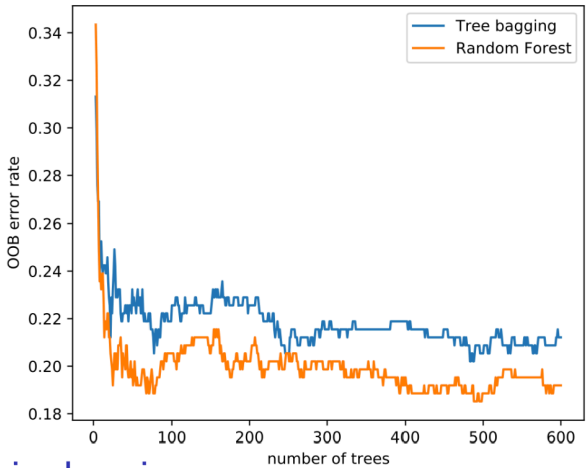

Figure 6

As shown in Figure 6, the **out-of-bag (OOB) error rate** decreases rapidly as the number of trees increases for both methods. The experiment is based on the *Heart Disease* dataset, where the goal is to predict whether a patient has heart disease (AHD = Yes/No) using medical predictors such as age, cholesterol, and maximum heart rate. The code trains two ensemble models:

- A **bagging classifier**, where each tree is built from a bootstrap sample of the training data.
- A **random forest classifier**, which extends bagging by randomly selecting a subset of predictors at each split (`max_features="sqrt"`).

Each model is trained incrementally (`warm_start=True`) from 3 to 600 trees, and the **OOB score** (an internal cross-validation estimate) is tracked at every step.
The **random forest** (orange line) achieves a lower OOB error than bagging (blue line) because random

feature selection reduces tree correlation, leading to lower variance and improved generalisation. This improvement arises because random feature selection decorrelates the trees, reducing the ensemble's overall variance.

Example: Comparison of classification performance using confusion matrices. The figure on the right compares how bagging and random forest predict heart disease:
- The **top confusion matrix** corresponds to **bagging**. It correctly classifies 80% of patients without heart disease and 77% of those with it.
- The **bottom confusion matrix** corresponds to the **random forest**, which slightly improves accuracy to 83% and 78% respectively.
These results confirm that random forests enhance predictive power by combining decorrelated trees, producing more stable and generalisable predictions than simple bagging.

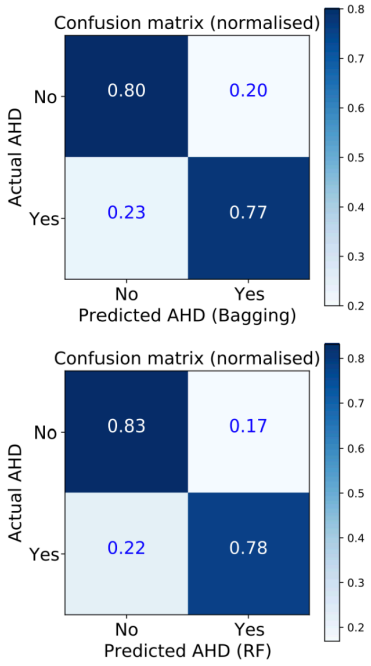

Figure 7: Confusion matrices for bagging (top) and random forest (bottom).

2.5 Variable importance

- Bagging and random forests have **high predictive power**, but the resulting ensemble of trees is difficult to interpret visually.
- To recover interpretability, we define an **overall measure of variable importance** that quantifies how much each predictor contributes to model performance.
- For **regression**, we record the total reduction in **Residual Sum of Squares (RSS)** due to splits involving a predictor X_j.
- For **classification**, we record the total decrease in **Gini index** or **deviance** associated with each predictor.
- Another way to assess importance is by **randomly permuting** a given predictor among the **out-of-bag (OOB) observations** and measuring how much the model's prediction error increases. A strong increase indicates that the variable was important for accurate predictions.

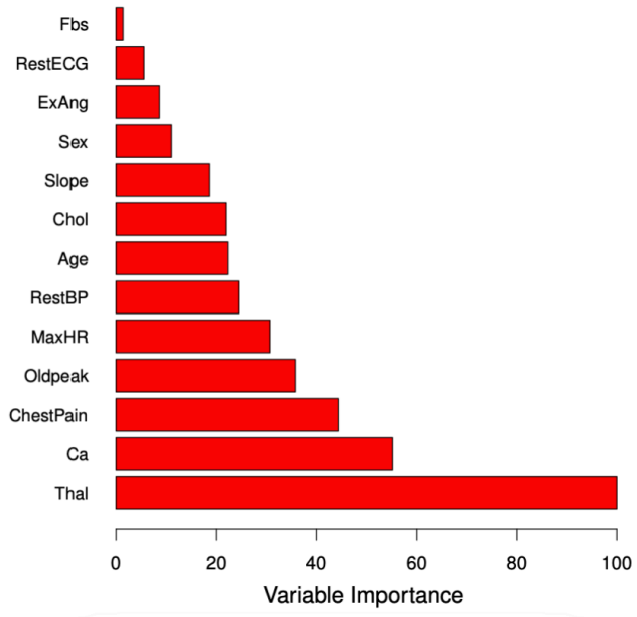

Figure 8: Variable importance in the heart disease dataset

As seen in Figure 8, the variables *Thal* (thalassemia type) and *Ca* (number of major vessels) are the most influential predictors of heart disease, followed by *ChestPain* and *Oldpeak*. Features such as *Fbs* (fasting blood sugar) or *RestECG* contribute comparatively little to the model's performance.
This ranking provides a powerful interpretative tool to understand which features drive the predictive accuracy of the random forest, even though the individual trees themselves are no longer easily interpretable.

2.6 Random forests and nearest-neighbours

Random forests can be understood as an **adaptive weighted nearest-neighbours method**, where "closeness" is defined by how often two observations fall into the same leaf nodes across trees.

- Recall that trees in a random forest are **fully grown and unpruned**. Each terminal node (leaf) represents a small region of the feature space containing similar training observations.
- For a new input x_0, each tree in the forest assigns x_0 to one of these regions, and the predicted response y_0 is the average of the y_i values within that leaf.
- Thus, for each tree, the prediction for x_0 is associated with one or more "neighbouring" training observations x_i, analogous to **nearest-neighbour methods** — except that here, *proximity is defined by the structure of the tree*, not by Euclidean distance.

- Each observation x_i contributes to the final prediction with a weight w_i proportional to how often it appears in the same terminal node as x_0 across the ensemble. Formally, the random forest prediction is

$$\hat{y}_0 = \sum_{i=1}^{n} w_i y_i, \quad \text{where } w_i \geq 0$$

  and w_i represents the **fraction of trees** where x_i and x_0 fall into the same leaf.

Hence, random forests can be viewed as a **smoothing method** that adapts locally to the data.
Unlike standard k-NN, the notion of "neighbourhood" is **data-driven and complex**, determined by the hierarchical partitioning of the feature space in multiple random trees.

2.7 Pros and cons of random forests

Random forests extend bagging by introducing randomness in feature selection, resulting in strong predictive models that generalise well to unseen data.
They are highly practical because they combine **robustness**, **accuracy**, and **low tuning requirements**, making them one of the most reliable ensemble methods in practice.

Pros:

- **Excellent predictive performance:** combining many uncorrelated trees drastically reduces variance and improves accuracy on both regression and classification tasks.
- **Minimal tuning required:** only a few parameters (e.g., number of trees B, number of features m per split) need to be set, and performance is typically stable across a wide range of values.
- **Built-in validation:** the **out-of-bag (OOB) error estimate** provides an internal measure of test error, eliminating the need for cross-validation.
- **Variable importance:** random forests naturally provide a measure of which predictors contribute most to the model's performance, aiding interpretability at the feature level.
- **Ease of implementation:** readily available in standard libraries such as *scikit-learn* and *R's randomForest*, and relatively fast to train even with large datasets.

Cons:

- **Loss of interpretability:** by averaging hundreds of trees, the model becomes a "black box." Although we can rank variable importance, we lose the intuitive visualisation and explanation that a single decision tree offers.
  This makes random forests less suitable for contexts where transparency and explainability are essential, such as legal or medical decision-making.

2.8 Digit classification with random forest

As seen in Figure 9, the **digit classification** task involves recognising handwritten digits (0–9) based on pixel intensity values.
Each model is trained on the same dataset and evaluated on its ability to correctly classify unseen digits.

Figure 9: Examples of handwritten digits used in the classification task.

Models compared:

- **linreg:** direct linear regression.
- **LDA1:** Linear Discriminant Analysis (LDA) based on the values x_i.
- **LDA2:** LDA based on x_i and x_i^2.
- **QDA:** Quadratic Discriminant Analysis with a distinct covariance matrix for each class.
- **log:** logistic regression (multinomial).
- **log-ridge:** logistic regression with ridge penalty.
- **log-lasso:** logistic regression with lasso penalty.
- **RF:** random forest.

Training and test errors (%)

Method	Training	Test
linreg	7.6	13.1
LDA1	6.2	11.5
LDA2	3.9	10.2
QDA	1.8	13.5
log	0.01	11.1
log-ridge	4.1	9.0
log-lasso	2.7	8.8
RF	0.0	5.9

The results show that **random forests** achieve the lowest **test error (5.9%)**, outperforming all other methods.
This strong generalisation comes from combining many decision trees trained on bootstrapped samples with random subsets of features. thereby creating a model structure that captures **non-linear dependencies** and **complex feature interactions** across the pixel space.

This strong generalisation arises from combining many decision trees trained on bootstrapped samples with random feature subsets, creating a structure that captures **non-linear dependencies** and **complex feature interactions** across the pixel space.

Regularised logistic models (ridge and lasso) also perform well, as they prevent overfitting by penalising large coefficients.
However, random forests remain the most powerful approach here, thanks to their ability to model intricate decision boundaries while maintaining robustness through averaging and decorrelation.

2.9 Digit classification: variable importance

As shown in Figure 10, random forests allow us to interpret complex models by computing a measure of **variable importance**, in this case, identifying which **pixels** contribute most to distinguishing each

handwritten digit.

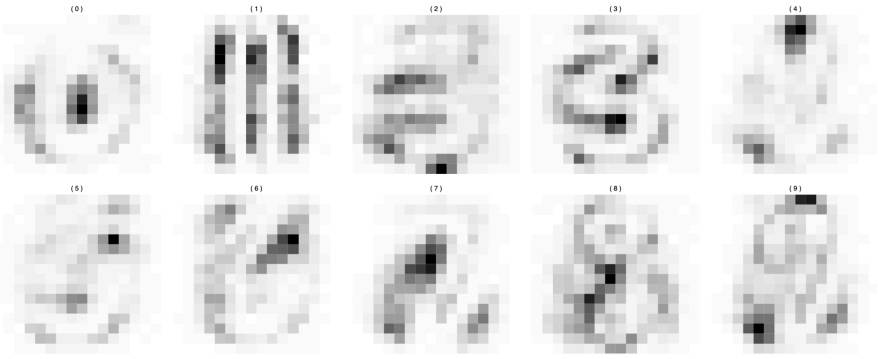

Figure 10: Pixel-level variable importance for each digit in the random forest model. Darker areas indicate higher importance.

Each heatmap corresponds to one digit (0–9), where the **dark regions** highlight the pixels most influential in the model's decisions.
For example, the central pixels are highly important for distinguishing **0** and **8**, whereas the **vertical strokes** dominate the importance maps for **1**.
These visual patterns align with how humans recognise digits, showing that the model learns to focus on the structural areas that define each number.

This technique provides a useful diagnostic tool:

- It helps confirm that the random forest has learned meaningful visual features rather than random noise.
- It also offers limited interpretability despite the ensemble's overall "black-box" nature, giving insights into what drives the model's predictions.

Thus, pixel-level variable importance visualisation bridges the gap between **performance** and **explainability**, showing how random forests internally differentiate complex visual classes.

3 Boosting

3.1 Boosting for trees

Boosting builds upon the idea of bagging but reverses its focus. In **bagging**, we use many **unbiased**, high-variance models (deep trees) and reduce their variance through averaging. Each tree is grown

independently on a bootstrap sample[2] of the data, and their predictions are averaged to form a stable, low-variance model.

Boosting, in contrast, aims to reduce the **bias** of the model. Instead of combining many deep, independent trees, it combines many **weak**, biased learners (typically shallow trees) that individually have low variance. These trees are not grown independently but **sequentially**: each new tree focuses on the parts of the data that the previous ones predicted poorly.

In other words:

- Bagging: reduces **variance** by averaging large, independent trees.
- Boosting: reduces **bias** by adding small trees that correct previous errors.
- Each tree in boosting learns from the **residuals** of the preceding model, gradually improving performance.
- This sequential correction makes boosting a form of *iterative learning*, where each step targets what remains unexplained.

3.2 Boosting algorithm for regression trees

The boosting algorithm formalises this process for regression trees. It builds the model step by step, with each new tree improving upon the errors of the last. Before starting, we fix three tuning parameters:

- **Number of repetitions** $B \in \mathbb{N}$: the number of trees (iterations).
- **Number of splits** $d \in \mathbb{N}$: the depth of each tree, controlling its complexity.
- **Shrinkage parameter** $\lambda \in (0, 1)$: a learning rate that controls how strongly each tree contributes to the model. $\lambda = 1$ no shrinkage; fast learning but high risk of overfitting. With λ too close to 0, learning becomes very slow, as each tree makes only a tiny adjustment.

Given a training set (x_i, y_i), $i = 1, \dots, n$, the algorithm proceeds as follows:

1. **Initialisation**: start with a null model and set the initial residuals to the observed outcomes:

$$\hat{f}(x) = 0, \quad r_i = y_i.$$

   This means the model initially predicts zero for all observations. So the residuals are numerically equal to the observed responses y_i, because the model hasn't learned anything yet. That's why the first residuals are sometimes described as the original response values or the real observations.

[2] A bootstrap sample is a dataset obtained by **sampling with replacement** from the original training data. Each decision tree in bagging is trained on a different bootstrap sample, which introduces randomness: although all trees see roughly the same number of observations, they each **contain different subsets and repetitions** of the data. This variability makes the trees slightly different from one another, allowing their predictions to be **averaged** to reduce variance in the final ensemble. Averaging smooths out these fluctuations, it reduces variance, but does not change the underlying model's bias (each tree is still an unbiased estimator, just noisy).

2. **Iterative fitting** For $b = 1, \ldots, B$:

   - Fit a regression tree $\hat{f}^b$ with d splits to the data (x_i, r_i). Each tree now tries to explain the residuals: the part of y_i that remains unexplained by the model so far.
   - Update the model by adding a *shrunken* version of this tree:

     $$\hat{f}(x) \leftarrow \hat{f}(x) + \lambda \hat{f}^b(x).$$

     The shrinkage factor λ prevents overfitting by slowing down the learning process.
   - Update the residuals to reflect the remaining error after adding the new tree:

     $$r_i \leftarrow r_i - \lambda \hat{f}^b(x_i).$$

     Residuals that remain large indicate regions where the model still performs poorly, guiding the next tree to focus on those points.

3. **Final model** After B iterations, the boosted model is the sum of all the shrunken trees:

   $$\hat{f}(x) = \sum_{b=1}^{B} \lambda \hat{f}^b(x).$$

   This model captures complex patterns by successively correcting its own mistakes — an effect that makes boosting powerful, yet prone to overfitting if B is too large.

In summary, boosting performs **slow learning** through small, sequential updates. While each tree alone is a poor predictor, together they form a strong model that reduces bias and achieves high accuracy when the tuning parameters are chosen carefully.

3.3 Tuning and learning rate in boosting

Boosting can be understood as a form of **slow learning**, since it builds the model step by step using many small trees that gradually improve $\hat{f}$ in regions where it performs poorly. Each tree makes only a small correction to the current model, focusing on residuals that remain large. Over successive iterations, these corrections accumulate into a powerful ensemble.

The **shrinkage parameter** λ further slows this process, controlling how much each new tree contributes. A smaller λ makes learning more gradual, allowing the algorithm to fit a wider variety of tree shapes to the residuals and avoid overfitting. This mechanism ensures that improvement happens progressively and in diverse parts of the predictor space.

Boosting has three key **tuning parameters**:

- **(a) Number of trees** B: determines how many iterations the algorithm performs.
  Unlike bagging and random forests, boosting can overfit if B becomes too large, although this typically happens slowly. The optimal B is chosen via cross-validation to stop learning before overfitting begins.

- **(b) Shrinkage parameter** λ: typically a small positive number (e.g. 0.01 or 0.001).
  It controls the *learning rate* of the algorithm. Smaller values imply slower learning but often lead to better generalisation, requiring more trees for good performance.
- **(c) Number of splits** d: controls the complexity of each individual tree.
  When $d = 1$, each tree is a **stump**, i.e. a single split. Such simple trees usually work surprisingly well because boosting reduces bias sequentially, making deep trees unnecessary.

Boosting can also be applied to **classification** problems, although the algorithm's formulation is slightly more complex than for regression.

3.4 Hitters data: boosting

To illustrate the behaviour of boosting, consider its application to the **Hitters** dataset, where the goal is to predict baseball players' salaries. The plots below show the **test error** as a function of the number of boosting iterations B for different values of the shrinkage parameter λ and tree depth d.

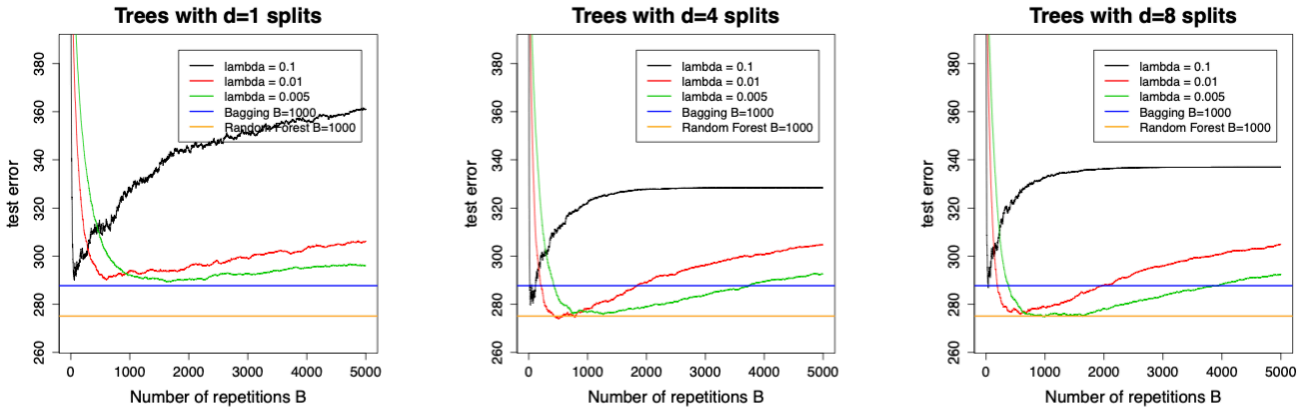

Figure 11: Test error as a function of the number of boosting iterations B for different values of λ and tree depths $d = 1, 4, 8$.

When $\lambda = 0.1$, the model learns very quickly, and the test error drops sharply but also rises again soon, which is a sign of **overfitting**. A small λ (e.g. 0.01 or 0.005) slows learning, resulting in a more gradual decrease of the test error that remains low over a broader range of B. This makes model selection via cross-validation more stable.

With **stumps** ($d = 1$), the model initially underfits, but increasing the depth to $d = 4$ allows it to reach lower test errors comparable to those of bagging and random forests. Further increasing d (e.g. to 8) offers little benefit and can even worsen performance, as overfitting begins earlier.

In practice, a **small learning rate** (around 0.01) and **moderately shallow trees** (e.g. $d = 4$) tend to yield the best trade-off between bias reduction and generalisation. Boosting thus provides a flexible yet controlled way to improve predictive performance through slow, sequential learning.

4 Support Vector Machines: Maximal Margin Classifier

Support Vector Machines (SVM) represent a geometric approach to classification. Unlike LDA or logistic regression, which model statistical relationships between Y and X, SVMs make **no distributional assumptions**. They aim instead to find a **decision boundary** (a hyperplane) that separates classes in the feature space.

A **hyperplane** in $\mathbb{R}^p$ is defined as the set of all x satisfying the linear equation

$$\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = 0.$$

This boundary divides the predictor space into two half-spaces. For any observation z, the function $z = \beta_0 + \beta_1 z_1 + \cdots + \beta_p z_p$ indicates its position relative to the hyperplane: points with $f(\mathbf{z}) > 0$ belong to one class, and those with $f(\mathbf{z}) < 0$ to the other. Points exactly on the hyperplane satisfy $f(\mathbf{z}) = 0$.

The vector $\beta = (\beta_1, \dots, \beta_p)$ is called the **normal vector** because it is orthogonal to the hyperplane. Its direction determines the orientation of the separating boundary, and its length is usually normalised to one to ensure a unique parametrisation. The intercept β_0 shifts the hyperplane along the direction of β without changing its orientation.

To build intuition:

- Points with $|f(\mathbf{z})|$ large are far from the boundary.
- Points with $f(\mathbf{z}) \approx 0$ lie close to it and are more uncertain.
- Thus, the magnitude $|f(\mathbf{z})|$ can be interpreted as the **distance** from **z** to the hyperplane.

Now consider a binary classification problem with classes labelled $y_i \in \{-1, +1\}$.
A hyperplane **separates** the training data if it assigns positive values of $f(x_i)$ to all observations with $y_i = +1$ and negative values to all with $y_i = -1$. Formally,

$$y_i(\beta_0 + \beta^\top x_i) > 0, \quad \text{for all } i = 1, \dots, n.$$

If such a hyperplane exists, there are infinitely many possible ones, each perfectly separating the two classes but differing in position.

The **Maximal Margin Classifier (MMC)** resolves this ambiguity by choosing the separating hyperplane that **maximises** the **margin**, defined as the smallest distance between the hyperplane and any training observation. Denoting this distance by M, the optimisation problem is:

$$\max_{\beta_0, \beta, M} M \quad \text{s.t.} \quad \|\beta\| = 1,\ y_i(\beta_0 + \beta^\top x_i) \geq M \text{ for all } i.$$

This ensures all observations lie at least M units away from the separating boundary, yielding the widest possible margin.

The observations that lie exactly at this minimal distance, i.e., those that satisfy $y_i(\beta_0 + \beta^\top \mathbf{x}_i) = M$, called the **support vectors**. They define the position of the maximal margin hyperplane. Small changes to these points can shift the boundary, whereas non-support points have no effect.

Although conceptually elegant, the MMC has two key **limitations**:

1. It is **unstable**, as it depends heavily on the few support vectors near the boundary.
2. It only works when the classes are **linearly separable**. In most real-world data, perfect separation is impossible, motivating more flexible extensions such as the **Support Vector Classifier**, which allows for a *soft* margin.

SVMs therefore build upon this geometric idea, gradually introducing relaxation and nonlinearity to handle more complex data structures while maintaining the same underlying optimisation principle.

4.1 Hyperplanes: construction

A **hyperplane** in $\mathbb{R}^p$ is defined by the equation

$$\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p = 0.$$

This represents a $(p-1)$-dimensional flat surface that divides the feature space into two half-spaces.

The vector $\beta = (\beta_1, \ldots, \beta_p)$ is called the **normal vector**, as it is **orthogonal** to the hyperplane. Its direction indicates the orientation of the separating boundary, while the intercept β_0 determines its distance from the origin.

To ensure a unique parametrisation, we usually normalise the normal vector so that $\|\beta\| = 1$. Without this constraint, scaling both β_0 and β by the same constant would produce the same hyperplane but different parameter values.

For any point $\mathbf{z} \in \mathbb{R}^p$, the signed distance from the hyperplane is given by $|\beta_0 + \beta_1 z_1 + \cdots + \beta_p z_p|$.
- If this value is **positive**, **z** lies on one side of the hyperplane.
- If it is **negative**, **z** lies on the opposite side.
- If it equals **zero**, **z** lies exactly on the hyperplane.

As shown in Figure 12, the **blue line** represents the hyperplane in $\mathbb{R}^2$, while the **red arrow** depicts the normal vector that is orthogonal to it. The absolute value of the linear function corresponds to the **distance** between each point and the hyperplane, capturing how far an observation lies from the separating boundary.

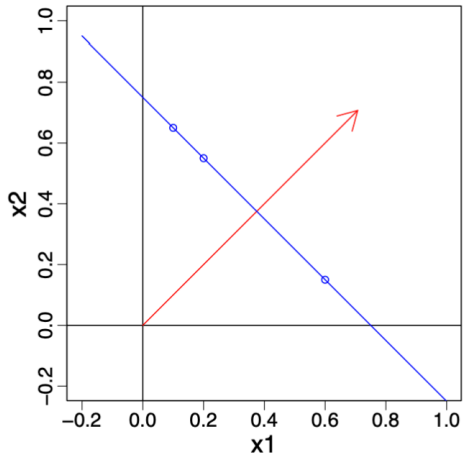

Figure 12: A hyperplane (**blue**) in $\mathbb{R}^2$ with its normal vector (**red**) orthogonal to it. The absolute value of the linear function gives the distance of each point from the hyperplane.

4.2 Separating hyperplanes

We now focus on the simplest SVM setting: two classes, denoted by $\mathcal{G} = \{-1, +1\}$.
Given training data $(x_1, y_1), \ldots, (x_n, y_n)$ with $y_i \in \{-1, +1\}$, we say that a **hyperplane is separating** if it correctly classifies all observations according to

$$\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} > 0 \text{ if } y_i = +1, \quad \beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip} < 0 \text{ if } y_i = -1.$$

These two inequalities can be written more compactly as

$$y_i(\beta_0 + \beta^\top \mathbf{x}_i) > 0, \quad i = 1, \ldots, n.$$

This expression captures both cases simultaneously: if $y_i = +1$, the term remains positive, while if $y_i = -1$, it reverses sign to ensure that the point lies on the opposite side of the hyperplane.

A separating hyperplane therefore divides the feature space so that all points of one class lie on one side and all points of the other class lie on the opposite side. When such a hyperplane exists, there are **infinitely many** possible separating boundaries, each producing zero training error but potentially different generalisation properties.

As illustrated in Figure 13, several hyperplanes can perfectly separate the same data. The plot on the left shows multiple valid separating lines, all achieving perfect classification, while the right panel depicts one such decision boundary dividing the feature space into two regions corresponding to the two classes.

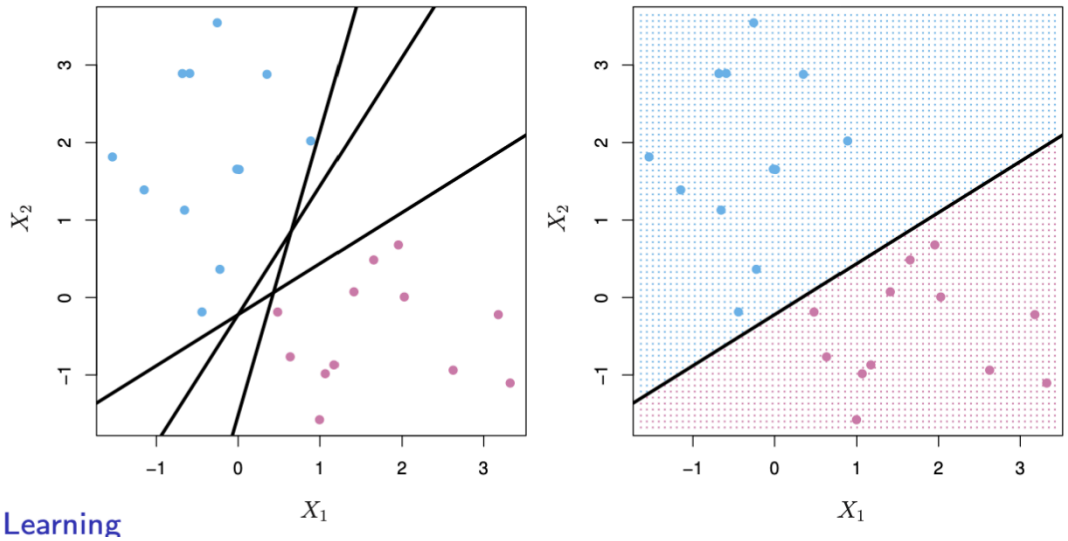

Figure 13: Different possible separating hyperplanes for linearly separable data.

4.3 Maximal Margin Classifier

When a separating hyperplane exists, it defines a **natural classifier** for new data points. The function $f(x_0)$ tells us on which side of the hyperplane a point x_0 lies: if it is positive, we assign the class $+1$, and if negative, the class -1. The **magnitude** of this value indicates how far the point lies from the boundary.

The **margin** represents the smallest distance between the training data and the separating hyperplane. The **Maximal Margin Classifier (MMC)** selects the hyperplane that maximises this margin, thereby creating the **widest possible gap** between the two classes. A wider margin leads to a more robust classifier, as small changes in the data are less likely to alter the decision boundary.

Only the observations lying exactly on the margin determine the boundary. These are known as **support vectors** because they "hold" the hyperplane in place. If one of them moves, the hyperplane shifts as well. Points further from the margin have no influence on its position. The MMC is obtained by solving a convex quadratic optimisation problem that efficiently identifies the separating hyperplane with the largest margin.

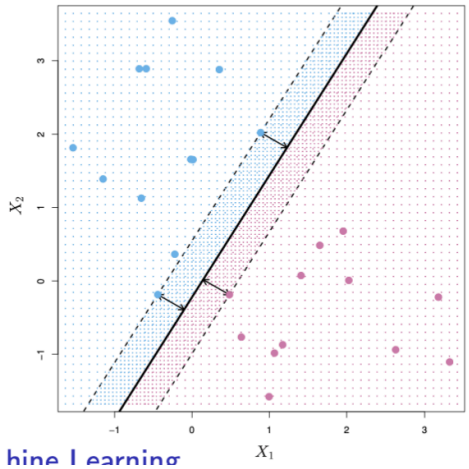

Figure 14: MMC: the optimal hyperplane (solid line) maximises the distance to the nearest training points of either class, defining the margin (dashed lines).

4.4 Comments and limitations

The optimisation problem behind the **maximal margin classifier** is a **convex quadratic programme**, which guarantees that the global optimum can be found efficiently.

However, as seen in Figure 14, only a few observations (the **support vectors**) determine the position of the separating hyperplane. Slightly moving one of these support vectors alters the optimal boundary, while moving other observations has no effect at all. This dependence on just a few critical points makes the classifier **sensitive** and potentially **unstable**.

In practice, this instability is compounded by another limitation: **most datasets are not perfectly linearly separable**. As illustrated in Figure 14, the maximal margin classifier only works when a hyperplane can fully separate the classes. When such perfect separation is impossible, the optimisation problem has no feasible solution.

These challenges motivate the next step, i.e., the **Support Vector Classifier**, which introduces a *soft margin* allowing for some classification errors while maintaining the geometric intuition of maximising the margin.

5 Support Vector Machines: Support Vector Classifier

5.1 Support Vector Classifier

The maximal margin classifier seeks a separating hyperplane that classifies the training data perfectly, which is often too rigid in the presence of noise or mislabelled observations. The support vector classifier is a *soft margin* method that permits controlled violations of the margin to improve robustness and generalisation.

We introduce slack variables $\xi_i \geq 0$ for training observations (x_i, y_i) with $y_i \in \{-1, +1\}$ and solve the optimisation problem:

$$\min_{\beta_0, \beta, \xi} \left(\frac{1}{2}\|\beta\|^2 + C \sum_{i=1}^{n} \xi_i \right) \quad \text{s.t.} \quad y_i(\beta_0 + \beta^\top x_i) \geq 1 - \xi_i,\ \xi_i \geq 0.$$

Here, the parameter $C > 0$ controls the trade-off between a wide margin (small $\|\beta\|$) and tolerance for violations (large ξ_i). In other words, the parameter C determines how strongly the model penalises violations of the margin (i.e., large ξ_i values). The variables ξ_i do not set the margin; instead, they measure how much each observation violates the margin constraint.
C controls *tolerance*, ξ_i measures *actual violations*, and β determines the *margin width*.

The positions of points are as follows:

- **Points with** $\xi_i = 0$: on or outside the margin.
- **Points with** $0 < \xi_i \leq 1$: inside the margin but correctly classified.
- **Points with** $\xi_i > 1$: misclassified.

As $C \to \infty$, we recover a hard margin (when separable). A smaller C widens the margin and typically yields a smoother, more stable decision rule focused on the most informative points — the *support vectors*.

Algorithmically:

1. Choose C (for instance, via cross-validation).
2. Solve the convex quadratic programme above.
3. Form the decision function $\hat{f}(x) = \text{sign}(\beta_0 + \beta^\top x)$, supported by training points with non-zero Lagrange multipliers (the support vectors).

Interpretation: the classifier allocates a *budget* of margin violations to gain robustness to outliers, achieve better average classification, and admit solutions when the classes are not perfectly separable.

As seen in Figure 15, the **left-hand plot** shows the *maximal margin classifier*, which fits the training data perfectly but results in a narrow margin that is highly sensitive to outliers. Even one mislabelled point could completely change the orientation of the separating hyperplane.

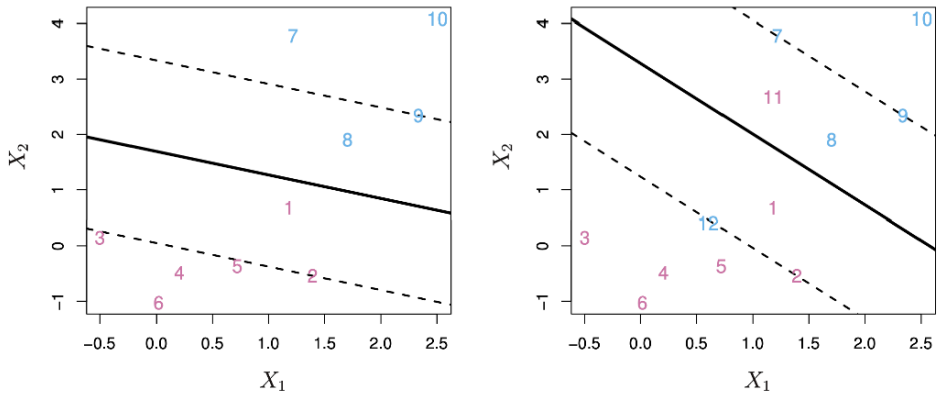

Figure 15: Hard- versus soft-margin solutions on the same dataset.

The **right-hand plot** depicts the *support vector classifier*, where some points are allowed to lie inside or even beyond the margin (those with $\xi_i > 0$). This flexibility leads to a wider margin and a more stable decision boundary.
The support vectors (the points closest to or inside the margin) determine this boundary, while all other points play no direct role in its position.

5.2 Mathematical formulation of the support vector classifier

The soft-margin formulation can also be expressed as a maximisation problem. We aim to find the largest possible margin M while allowing controlled violations through slack variables $\xi_i \geq 0$.

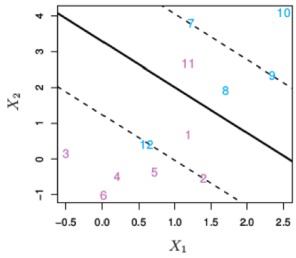

Figure 16: Soft-margin classifier with slack variables ξ_i and total budget b.

The optimisation problem is:

$$\max_{\beta_0,\ldots,\beta_p,\ \xi_1,\ldots,\xi_n} M$$

subject to

$$\sum_{j=1}^{p} \beta_j^2 = 1, \quad y_i(\beta_0 + \beta_1 x_{i1} + \cdots + \beta_p x_{ip}) \geq M(1 - \xi_i),$$

$$\xi_i \geq 0, \quad \sum_{i=1}^{n} \xi_i \leq b.$$

As illustrated in Figure 16, the soft-margin classifier allows some points to fall inside or even beyond the margin, depending on their corresponding ξ_i values.
Each ξ_i quantifies how much observation i violates the margin constraint:

- $\xi_i = 0$: the observation lies on or outside the margin (correctly classified).
- $0 < \xi_i \leq 1$: the observation lies inside the margin but is still correctly classified.
- $\xi_i > 1$: the observation is misclassified, on the wrong side of the separating hyperplane.

In Figure 16, the points exactly on the margin correspond to $\xi_i = 0$, while those inside the shaded margin regions have $0 < \xi_i \leq 1$.
Points that cross the decision boundary entirely (on the wrong side) represent $\xi_i > 1$ and contribute most to the violation term in the optimisation.

The constant $b > 0$ acts as a *tuning* or *regularisation* parameter that sets the total "budget" of margin violations across all n observations. A larger b allows more flexibility, useful for noisy or non-separable data, while a smaller b enforces stricter separation with fewer violations.

Here, the *support vectors* are the observations that either lie exactly on the margin or violate it (i.e. those for which $\xi_i > 0$). Only these points determine the position and orientation of the separating hyperplane ; all other observations, being further from the decision boundary, have no influence on the classifier.

Consequently, the regularisation parameter b indirectly controls the number of support vectors and hence the complexity of the model. A smaller b (stricter margin) limits violations, reducing the number of support vectors and yielding a simpler, higher-bias but lower-variance model. Conversely, a larger b allows more margin violations, increasing the number of support vectors and producing a more flexible, lower-bias but higher-variance classifier.

5.3 Tuning parameter b

The regularisation constant b plays a central role in controlling the flexibility of the support vector classifier. It sets the total "budget" of margin violations permitted across all training observations, and therefore determines the number of support vectors used to define the decision boundary.

As illustrated in Figure Figure 17, increasing or decreasing b directly influences the width of the margin and the bias–variance characteristics of the model:

- **Large** b allows more violations, leading to **wider margins** and a **higher-bias, lower-variance** classifier. The decision boundary becomes smoother and less sensitive to individual observations.
- **Small** b enforces fewer violations, producing **tighter margins** and a **lower-bias, higher-variance** classifier. The boundary fits the training data more closely but risks overfitting.

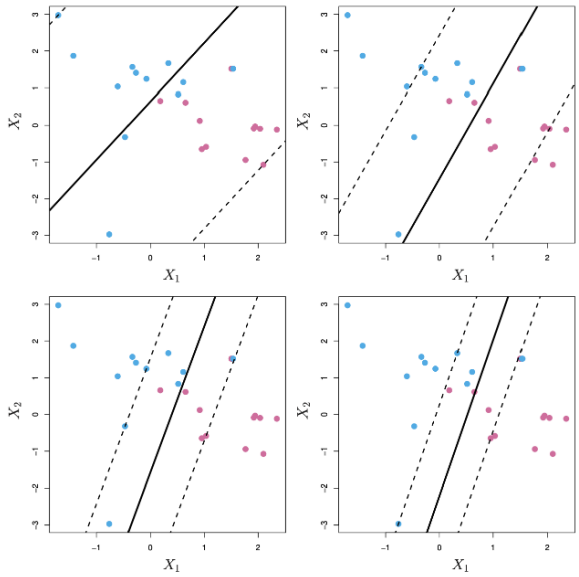

Figure 17: Effect of the tuning parameter b on the margin width and classifier flexibility.

The optimal value of b is typically chosen through **cross-validation**, balancing the trade-off between margin width and classification accuracy on unseen data.

5.4 Limitations of the support vector classifier

Although the support vector classifier is a powerful method for constructing linear decision boundaries, it has an important limitation: it can only produce *linear* separation in the feature space.

As shown in Figure 18, when the true decision boundary is non-linear, a linear classifier (such as the support vector classifier) fails to capture the complex shape that separates the two classes. In such cases:

- A **linear decision boundary** performs poorly if the data are not linearly separable in the feature space $(X_1, \ldots, X_p)$.
- One might try to **enlarge the feature space**, for example by adding polynomial terms such as $X_1^2, X_2^2, \ldots$, and then apply a linear classifier in this higher-dimensional space.
- However, enlarging the feature space dramatically increases dimensionality, and the resulting **computational cost** can become infeasible for large datasets.

This limitation motivates the next step: introducing **non-linear decision boundaries** through the use of kernels, which implicitly map the data into a higher-dimensional space without the need to compute the expanded features explicitly.

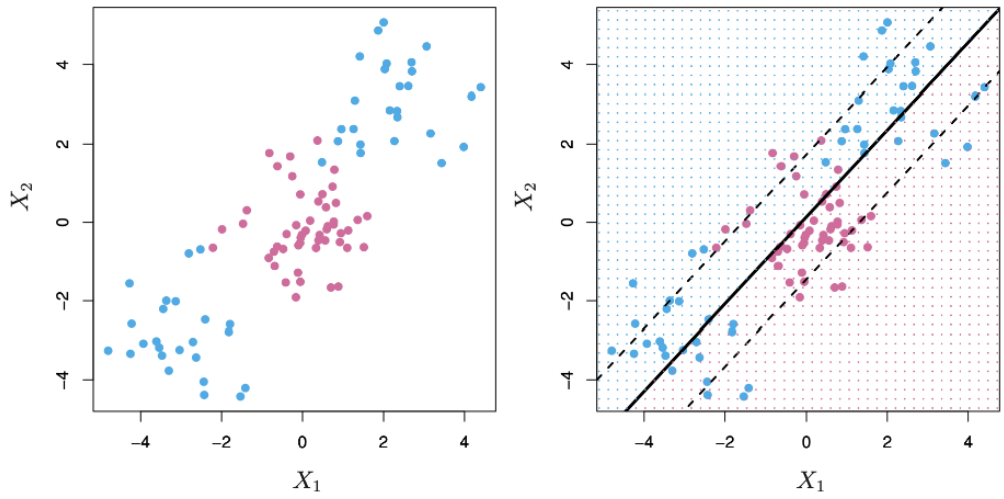

Figure 18: Limitation of the support vector classifier: poor performance with non-linear class boundaries.

6 Support Vector Machines

6.1 Support Vector Machines

Support Vector Machines (SVMs) generalise the support vector classifier to allow **non-linear decision boundaries**. They achieve this by enlarging the feature space — but in an *automatic* way, without explicitly constructing new variables as in manual feature engineering.

In the support vector classifier, we used the decision function

$$f(x) = \beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p,$$

which defines a **linear hyperplane** in the feature space. Once the optimal parameters are found, this function is used to classify new observations:

$$\hat{G}(x) = \begin{cases} +1 & \text{if } f(x) > 0, \\ -1 & \text{if } f(x) < 0. \end{cases}$$

A deep mathematical result shows that the optimal separating function can be rewritten as

$$f(x) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i \langle x, x_i \rangle,$$

where each α_i is a parameter associated with training observation i, and $\langle \mathbf{x}, \mathbf{z} \rangle = \sum_{j=1}^{p} x_j z_j$ denotes the **inner product** between two feature vectors.

Only a small subset of training points have $\alpha_i \neq 0$; these correspond to the **support vectors** that determine the decision boundary. All other points do not influence the classifier.

The inner product $\langle x, z \rangle$ can be interpreted as a **similarity measure**: it is large when two vectors point in similar directions (highly correlated) and small when they are nearly orthogonal. This observation motivates the idea of **replacing** the inner product with a more general similarity function $K(\mathbf{x}, \mathbf{z})$, called a **kernel**.

By doing so, the Support Vector Machine retains the same structure as the support vector classifier but gains the ability to create **non-linear decision boundaries** in the original predictor space.

6.2 The Kernel Trick

Support Vector Machines classify observations using a function that is, in general, *non-linear* in the original feature space:

$$f(\mathbf{x}) = \beta_0 + \sum_{i=1}^{n} \alpha_i y_i K(\mathbf{x}, \mathbf{x}_i),$$

where $K(\mathbf{x}, \mathbf{z})$ is a *(positive definite)* **kernel function** measuring the similarity between two feature vectors $\mathbf{x}, \mathbf{z} \in \mathbb{R}^p$.

By replacing the inner product $\langle \mathbf{x}, \mathbf{z} \rangle$ with a kernel $K(\mathbf{x}, \mathbf{z})$, the SVM effectively **enlarges the feature space implicitly**, without ever computing the higher-dimensional representation explicitly.
This elegant idea is known as the **kernel trick**.

- With the **linear kernel**, $K(\mathbf{x}, \mathbf{z}) = \langle \mathbf{x}, \mathbf{z} \rangle$, we recover the standard support vector classifier.
- With **non-linear kernels**, the classifier defines curved and flexible decision boundaries in the original predictor space while remaining linear in the (implicit) high-dimensional feature space.

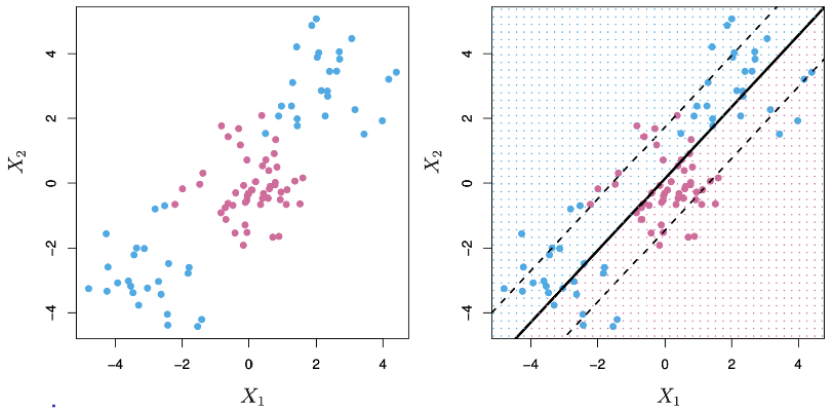

Figure 19: Illustration of the kernel trick: the SVM implicitly enlarges the feature space to achieve non-linear decision boundaries.

As seen in Figure 19, mapping the data into a higher-dimensional space allows a simple linear boundary in that space to correspond to a **non-linear boundary** in the original feature space, i.e., the essence of the kernel trick.

6.3 The Kernel Trick: Examples

Support Vector Machines can use different kernels to implicitly map the data into higher (even infinite) dimensional spaces, enabling highly flexible decision boundaries.

6.3.1 The Polynomial Kernel

The **polynomial kernel** is defined as

$$K(\mathbf{x}, \mathbf{z}) = \left(1 + \gamma \sum_{j=1}^{p} x_j z_j\right)^d,$$

where γ is a scaling parameter and d is the degree of the polynomial.

This kernel corresponds to fitting a **support vector classifier in a higher-dimensional space** that includes all polynomial terms up to degree d.
In the code, this is specified as `svm = SVC(kernel='poly', degree=d, gamma=gamma)`.
The parameter γ controls the **scale** of the transformation and hence the flexibility of the decision boundary.

- Larger γ and higher d produce more flexible, curved boundaries that adapt to complex patterns.
- Smaller γ or lower d result in smoother, more regular decision surfaces.

6.3.2 The Radial (Gaussian) Kernel

The **radial basis function (RBF)** or **Gaussian kernel** is defined as

$$K(\mathbf{x}, \mathbf{z}) = \exp\left(-\gamma \sum_{j=1}^{p} (x_j - z_j)^2\right),$$

where $\gamma > 0$ determines how quickly similarity decays with distance.

This kernel corresponds to fitting a **support vector classifier in an infinite-dimensional feature space**, since the associated basis functions are infinitely many.
In the code, it is specified with `kernel='rbf'` inside `svm = SVC(kernel='rbf', gamma=gamma)`, allowing γ to control how local or global the decision surface becomes.

- Large values of γ make the kernel highly localised: only nearby observations influence the classification, producing **flexible, non-linear** decision boundaries.
- Small values of γ yield smoother, more global fits, where distant points still contribute, resulting in **wider margins** and more stable boundaries.

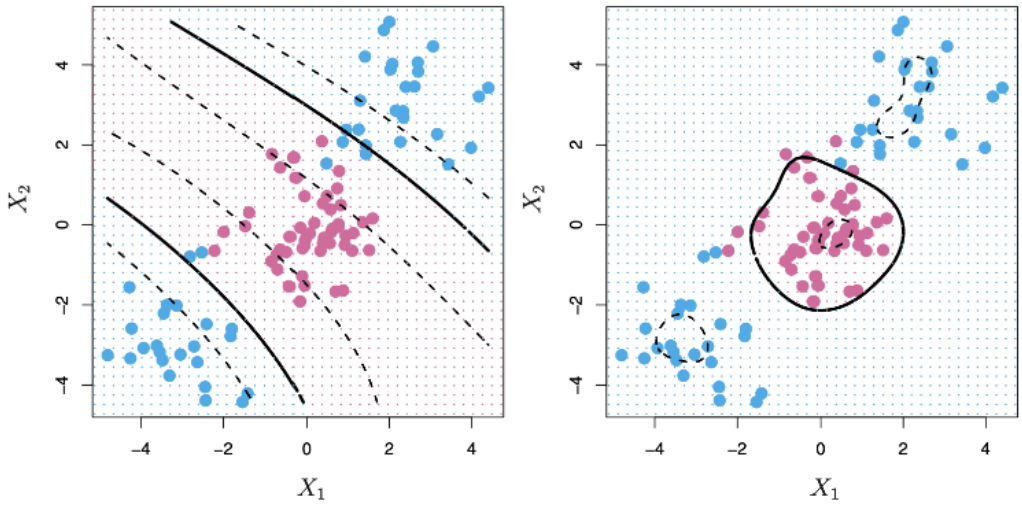

Figure 20: Polynomial and radial kernels producing different decision boundaries.

As seen in Figure 20, the **left-hand plot** based on the polynomial kernel captures some curvature but remains relatively smooth, while the **right-hand plot**, using the radial kernel, displays a highly adaptive boundary that separates non-linear patterns effectively. This demonstrates how the choice of kernel and the tuning of γ determine the **locality** and **flexibility** of the Support Vector Machine.

6.4 Comments on Support Vector Machines

Support Vector Machines (SVMs) are a **powerful and flexible tool** for classification, capable of learning both linear and non-linear decision boundaries through the use of kernels.

Although the underlying mathematics are complex, the **kernel trick** allows us to apply SVMs without needing to explicitly compute the high-dimensional feature mappings. In practice, fitting an SVM simply requires computing the pairwise similarities

$$K(\mathbf{x}_i, \mathbf{x}_j)$$

for all distinct pairs of training observations (i, j).

A wide variety of kernels have been developed, but even the **standard choices**, such as the linear, polynomial, and radial basis (Gaussian) kernels, typically perform very well across a broad range of applications.

For **multi-class problems**, SVMs can be extended using:

- **One-versus-one classification**, where a separate classifier is trained for each pair of classes; or
- **One-versus-all classification**, where each classifier distinguishes one class from all the others.

For further discussion and examples, see Chapter 9 of *An Introduction to Statistical Learning (ISL)*.

6.5 Support Vector Machines in Python

The following Python example fits Support Vector Machines on the *Iris* dataset using an **RBF (radial basis function) kernel**. Only the first two features ("sepal length" and "sepal width") are used so that the resulting decision boundaries can be visualised in two dimensions.

The code iterates over three values of the tuning parameter γ (0.1, 1, and 20) each time fitting an SVM model and plotting its resulting decision boundary.

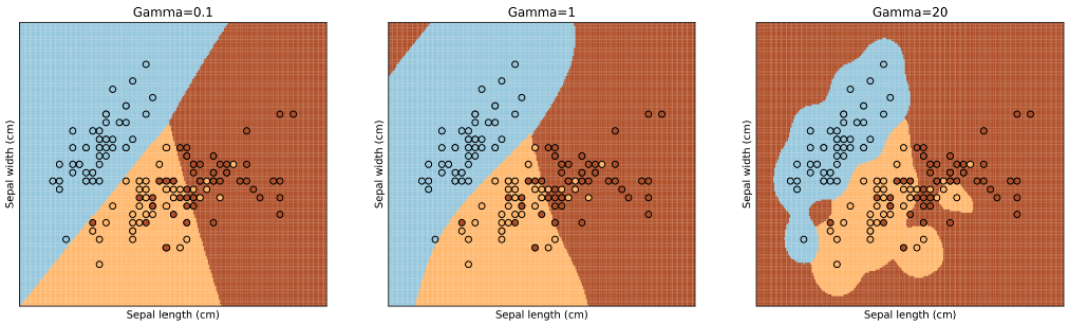

Figure 21: Decision boundaries of radial-kernel SVMs on the Iris dataset for different γ values.

In the code, the line `svm = SVC(C=1, kernel='rbf', gamma=gamma)` creates an SVM classifier with the specified γ, and the subsequent loop visualises the corresponding decision boundaries.

Each subplot in Figure 21 illustrates how γ controls the **locality** and **flexibility** of the model:

- In the **left-hand plot** ($\gamma = 0.1$), the kernel is *broad and global*: similarities decay slowly, so many training points influence each decision. The resulting boundary is smooth and less adaptive, i.e., a **high-bias, low-variance** model.
- In the **middle plot** ($\gamma = 1$), the decision surface adapts more closely to the data while preserving a reasonable margin. This corresponds to a **balanced bias–variance trade-off**.
- In the **right-hand plot** ($\gamma = 20$), the kernel is *highly localised*: only nearby points affect predictions. The decision boundary becomes extremely wiggly, indicating **low bias but high variance** and the risk of overfitting.

This example highlights the **bias–variance trade-off** governed by γ: smaller values yield smoother, more generalisable boundaries, whereas larger values produce complex, locally adaptive ones.

6.5.1 Effect of the Cost Parameter

In the next example, the SVM is again fitted on the *Iris* dataset using the **radial basis function (RBF) kernel**, this time fixing $\gamma = 20$ and varying the **cost parameter** C (the inverse of the margin budget).

The parameter C determines how strongly misclassifications are penalised during training:

- **Small** C allows more margin violations (a *larger budget*) leading to a smoother, more regularised decision boundary.
- **Large** C penalises violations heavily (a *smaller budget*) yielding a tighter boundary that fits the training data more closely.

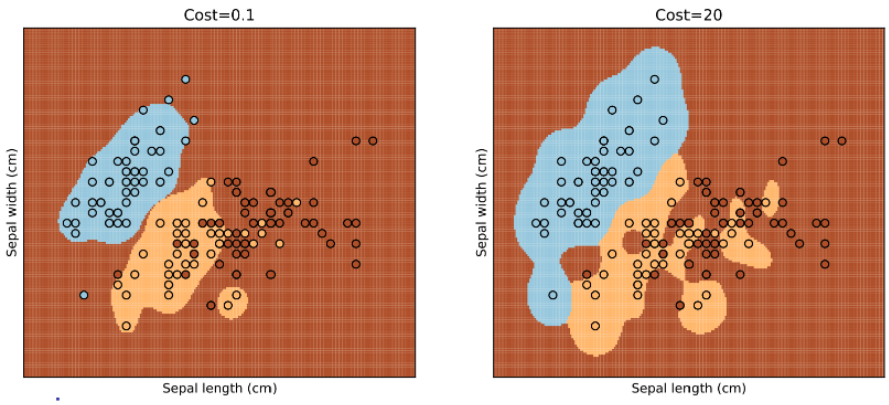

Figure 22: Decision boundaries for different values of the cost parameter C.

As seen in Figure 22, the **left-hand plot** with $C = 0.1$ results in a wider margin that tolerates a few misclassified points but generalises better. In contrast, the **right-hand plot** with $C = 20$ shows a much tighter boundary that fits almost all training data points correctly but risks overfitting.

This again illustrates the **bias–variance trade-off**: smaller C increases bias but reduces variance, while larger C decreases bias at the expense of higher variance.

6.5.2 Model Selection by Cross-validation

In this final example, an SVM with a **radial basis function (RBF) kernel** is tuned automatically using **cross-validation**.
Instead of choosing the parameters γ and C manually, a grid search is performed over a predefined range of values, and the model is evaluated with k-fold cross-validation to identify the best-performing combination.

As illustrated in Figure 23, the selected parameters ($C = 1.2$, $\gamma = 5$ in this case) yield a decision boundary that balances **flexibility** and **smoothness**. Compared with the earlier examples:

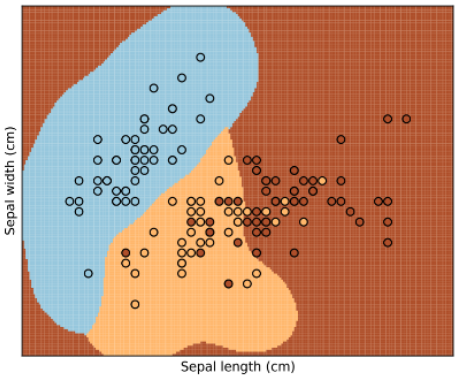

Figure 23: Optimal decision boundary obtained by tuning the parameters C and γ via cross-validation.

- The boundary is less rigid than with a small γ or small C, avoiding underfitting.
- It is also less wiggly than with excessively large values, preventing overfitting.

This process demonstrates the value of **cross-validation** for model selection: it systematically explores combinations of hyperparameters to achieve the best trade-off between bias and variance, ensuring robust generalisation on unseen data.

6.6 Digit Classification with SVMs

We now return to the **digit classification** problem introduced earlier (see Figure **?@fig-ML6-digit-class**).
Support Vector Machines are applied here to classify handwritten digits using a **One-Versus-One** scheme, where one binary classifier is trained for each pair of digits.
We compare SVMs using **linear**, **polynomial**, and **radial (Gaussian)** kernels. The tuning parameter C was selected by **10-fold cross-validation** to optimise test accuracy.

Training and test data:

- Training data $(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)$, $n = 7290$, are used for model fitting and parameter tuning.
  - Predictors $\mathbf{x}_i$ are greyscale pixel intensities of each image $\in \mathbb{R}^p$.
  - Outcomes y_i correspond to the true digit labels $\in \{0, 1, \ldots, 9\}$.
- Test data $(\hat{\mathbf{x}}_1, \hat{y}_1), \ldots, (\hat{\mathbf{x}}_m, \hat{y}_m)$, $m = 2006$, are used only for evaluating predictive performance.

Models compared:

- **linreg:** direct linear regression.
- **LDA1:** LDA based on the raw pixel values x_i.
- **LDA2:** LDA based on x_i and their squares x_i^2.
- **QDA:** Quadratic Discriminant Analysis with a separate covariance matrix per class.
- **log:** multinomial logistic regression.
- **log-ridge:** logistic regression with ridge penalty.
- **log-lasso:** logistic regression with lasso penalty.
- **RF:** random forest.
- **linear SVM:** support vector classifier with a linear kernel.
- **radial SVM:** support vector machine with an RBF kernel.

Training and test errors (%)

Method	Training	Test
linreg	7.6	13.1
LDA1	6.2	11.5
LDA2	3.9	10.2
QDA	1.8	13.5
log	0.01	11.1
log-ridge	4.1	9.0
log-lasso	2.7	8.8
RF	0.0	5.9
linear SVM	1.5	6.5
radial SVM	0.6	5.4

The results show that both **linear** and **radial** SVMs perform extremely well, with the **radial kernel** achieving the lowest test error (5.4%).
This demonstrates the flexibility of kernel-based SVMs in modelling complex, non-linear decision boundaries, even in high-dimensional spaces like image pixels.

As seen throughout this lecture, the **kernel trick** allows SVMs to implicitly map data into richer feature spaces without explicitly constructing new variables, resulting in powerful and computationally efficient classifiers.

7